

MINISTRY OF EDUCATION, CULTURE AND RESEARCH OF THE REPUBLIC OF MOLDOVA
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

Maxim Ormanji FAF-242

Report

Laboratory work No.2

Operating System

Topic: Meet the OS You Will Build (xv6)

Chișinău – 2026

Part 1. Build it and boot it

I cloned the xv6 repository, went into the folder and ran `make qemu`. The kernel booted and showed the `$` prompt.

```
vboxuser@Ubuntu:~/os-lab2$ git clone https://github.com/mit-pdos/xv6-riscv ~/xv6-riscv
vboxuser@Ubuntu:~$ cd ~/xv6-riscv
vboxuser@Ubuntu:~/xv6-riscv$ make qemu

xv6 kernel is booting

hart 2 starting
hart 1 starting
init: starting sh
$
```

Part 2. Use xv6 as the Unix it is

I ran the built-in programs: `ls`, `cat README`, `echo hello xv6`, `ls | grep c`, `wc README` (the output of `cat README` is shortened).

```
$ ls
.              1 1 1024
..             1 1 1024
README         2 2 2441
cat            2 3 36728
echo           2 4 35592
forktest       2 5 18136
grep           2 6 44184
init           2 7 35976
kill           2 8 35512
ln             2 9 35304
ls             2 10 42880
mkdir          2 11 35568
rm             2 12 35552
sh             2 13 58312
stressfs       2 14 36416
usertests      2 15 206800
grind          2 16 51824
wc             2 17 37728
zombie         2 18 34848
logstress      2 19 37560
forphan        2 20 36352
dorphan        2 21 35800
sync           2 22 34944
console        3 23 0
$ cat README
xv6 is a re-implementation of Dennis Ritchie's and Ken Thompson's Unix
Version 6 (v6).  xv6 loosely follows the structure and style of v6,
but is implemented for a modern RISC-V multiprocessor using ANSI C.

...
$ echo hello xv6
```

```
hello xv6
$ ls | grep c
cat            2 3 36728
echo           2 4 35592
wc             2 17 37728
sync           2 22 34944
console        3 23 0
$ wc README
48 336 2441 README
```

Observe (Part 2)

1. **List three programs xv6 ships with.**
   `ls`, `cat` and `echo`. The `ls` output also shows `grep`, `wc`, `mkdir`, `rm`, `kill`, `ln` and others.
2. **You just used a pipe (`|`) inside xv6. Which two OS features must exist for a pipe to work?**
   Processes and communication between processes. In `ls | grep c` two programs run (`ls` and `grep`), so the OS must be able to create processes. The output of `ls` has to reach the input of `grep`, so there must be a communication channel (a pipe) provided by the kernel.
3. **In one sentence: how does the xv6 shell compare to the Linux shell from Lab 1?**
   The xv6 shell is much simpler than the Linux shell, but it does the basics: it runs programs and supports pipes.

Part 3. Read the source

I quit QEMU and read `user/ls.c` and `user/cat.c`, looked at the list of system calls in `user/user.h`, and searched `kernel/sysfile.c` for `sys_read` and `sys_write`. Only the parts needed for the answers are shown below.

Beginning of `user/cat.c`:

```
vboxuser@Ubuntu:~/xv6-riscv$ sed -n "1,30p" user/cat.c
#include "kernel/types.h"
#include "kernel/fcntl.h"
#include "user/user.h"

char buf[512];

void
cat(int fd)
{
  int n;

  while ((n = read(fd, buf, sizeof(buf))) > 0) {
    if (write(1, buf, n) != n) {
      fprintf(2, "cat: write error\n");
      exit(1);
    }
  }
  if (n < 0) {
    fprintf(2, "cat: read error\n");
```

```
    exit(1);
  }
}

int
main(int argc, char *argv[])
{
  int fd, i;

  if (argc <= 1) {
    cat(0);
```

List of system calls in `user/user.h`:

```
vboxuser@Ubuntu:~/xv6-riscv$ cat user/user.h
// system calls
int fork(void);
int exit(int) __attribute__((noreturn));
int wait(int *);
int pipe(int *);
int write(int, const void *, int);
int read(int, void *, int);
int close(int);
int kill(int);
int exec(const char *, char **);
int open(const char *, int);
int mknod(const char *, short, short);
int unlink(const char *);
int fstat(int fd, struct stat *);
int link(const char *, const char *);
int mkdir(const char *);
int chdir(const char *);
int dup(int);
int getpid(void);
char *sys_sbrk(int, int);
int pause(int);
int uptime(void);
int sync(void);
...
```

Where `sys_read` and `sys_write` are implemented:

```
vboxuser@Ubuntu:~/xv6-riscv$ grep -n "sys_read\|sys_write" kernel/sysfile.c | head
69:sys_read(void)
83:sys_write(void)
```

Observe (Part 3)

1. **Which system calls does `user/cat.c` use, and what does each one ask the kernel to do?**
   - `read()` asks the kernel to read data from a file (by its descriptor) into a buffer.
   - `write()` asks the kernel to write data to the output (descriptor 1, the screen).
   - `exit()` asks the kernel to end the program.
   - `open()` and `close()` (in the second half of the file, in `main`) ask the kernel to open a

file and later close it.

2. **In which kernel file, and on which line, is `sys_read` implemented?**
   In `kernel/sysfile.c`, line 69 (`sys_write` is on line 83).
3. **In one sentence: what is the difference between the code in `kernel/` and the code in `user/`?**
   `kernel/` contains the operating system itself, which manages memory, files and processes, while `user/` contains ordinary programs (`ls`, `cat`, the shell) that ask the kernel for services through system calls.

Part 4. Write your first xv6 program

I created `user/sleep.c`:

```
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  if(argc != 2){
    fprintf(2, "usage: sleep <ticks>\n");
    exit(1);
  }
  pause(atoi(argv[1]));   // system call into the kernel
  exit(0);
}
```

Then I added `$U/_sleep` to the `UPROGS` list in the `Makefile` (beginning of the list):

```
UPROGS=\
        $U/_cat\
        $U/_echo\
        $U/_sleep\
...
```

After rebuilding with `make qemu`, I tested the program inside xv6:

```
vboxuser@Ubuntu:~/xv6-riscv$ make qemu
$ sleep 10
$ sleep
usage: sleep <ticks>
$
```

Note. In my version of xv6 the system call that puts a process to sleep is called `pause` (see `int pause(int);` in `user/user.h`), not `sleep` as in the lab text. That is why `sleep.c` calls `pause(atoi(argv[1]))`. The program itself is still named `sleep`.

Result: `sleep 10` pauses and then returns to the prompt, and `sleep` without an argument prints `usage: sleep <ticks>`.

Conclusion

I built and booted xv6, used it as a Unix system, saw how the kernel code (`kernel/`) is separated from user programs (`user/`), and wrote the `sleep` program, which makes a system call into a kernel that I built myself.